MLB PREDICTION PIPELINE

Shadow Ensemble v5.1 — Technical Architecture & Model Performance

Version 5.1 · April 2026

Three-algorithm shadow ensemble (XGBoost + LightGBM + CatBoost) blended via a Bayesian Hierarchical NUTS-MCMC Level-2 Stacker.
GPU-accelerated on RTX 5080 with CuPy Monte Carlo (50K trials).
Includes real fit statistics, AUC curves, calibration, and backtest ROI from actual 2023–2026 model output.

1. EXECUTIVE SUMMARY

The MLB Prediction Pipeline v5.1 ('The Wizard Report') is a fully-automated, multi-agent system producing daily game-by-game probability estimates and Kelly-staked betting signals for MLB run lines, moneylines, and totals. Version 5.1 introduces the Shadow Ensemble architecture: three independent tree algorithms (XGBoost, LightGBM, CatBoost) trained on identical data, with prediction spread used as an uncertainty signal. A Bayesian Hierarchical NUTS-MCMC stacker replaces the prior LogisticRegression(C=10) level-2 model. All training runs GPU-accelerated on an NVIDIA RTX 5080; the Monte Carlo simulation uses CuPy for all 50,000-trial Poisson draws.

Metric	Run Line Model	Moneyline Model	Totals Regressor
Architecture	XGB + LGBM + CatBoost shadow	XGB + LGBM + CatBoost	XGB multi-quantile
Validation year	2025 (n=2,398)	2025 (n=2,514)	2025 (n=2,398)
NCV Fold 1 AUC (XGB)	0.6056	0.6111	MAE = 3.4247 R
NCV Fold 2 AUC (XGB)	0.5869	0.5970	MAE = 3.5778 R
Val 2025 Raw AUC	0.5793	0.5642	RMSE = 4.5533 R
Val 2025 Platt AUC	0.5793	0.5642	—
Val 2025 Bayesian Stack	0.5760	—	—
Brier (Bayesian Stack)	0.2259	0.2453	—
Log-Loss (Bayesian Stack)	0.6436	0.6836	—
Accuracy @50%	63.9%	55.5%	—
2026 YTD AWAY ★★ ROI	+31.8% (n=84)	—	—

The model shows consistent positive ROI on AWAY +1.5 ★★ signals across 2023–2026. The 2026 YTD dominant signal is AWAY +1.5 ★★ (84 bets, +31.8% ROI as of April 2026). The Bayesian stacker's SP-handedness segmentation (LvL/LvR/RvL/RvR) provides additional calibration in matchup-specific contexts. The Three-Part Lock gate routes capital only through spots where the model and Pinnacle sharp lines agree AND the model shows meaningful edge over the retail market.

2. SHADOW ENSEMBLE ARCHITECTURE (V5.1)

Version 5.1 introduces three fully-trained tree models on every target. The XGBoost model is the OFFICIAL pipeline model that feeds the Level-2 Bayesian stacker. LightGBM and CatBoost run as shadow models — their predictions are used to compute model_spread (uncertainty signal) and are logged for future ensemble weighting experiments.

Model	Role	Algorithm	GPU Backend	Model Files
XGBoost	OFFICIAL — feeds Bayesian stacker	hist, depth=5, n_est=600	cuda / RTX 5080	xgb_rl.json, xgb_ml.json, xgb_total.json, xgb_rl_team.json
LightGBM	Shadow — uncertainty signal	leaf-wise, num_leaves=63	device=gpu / RTX 5080	lgb_shadow.json, lgbm_rl.pkl, lgbm_ml.pkl, lgbm_total.pkl
CatBoost	Shadow — uncertainty signal	symmetric trees, depth=6	task_type=GPU / RTX 5080	cat_shadow.cbm, cat_rl.pkl, cat_ml.pkl, cat_total.pkl

2.1 Uncertainty Signal: model_spread

```
# model_spread — computed at inference time:
model_spread = max(xgb_raw, lgbm_raw, cat_raw) – min(xgb_raw, lgbm_raw, cat_raw)
# High spread → models disagree → bet filtered or downweighted
# Low spread → all three models converge → higher confidence signal
```

model_spread is included as a stacking feature (ml_model_vs_vegas_gap) in the Bayesian level-2 model. Games with model_spread > 0.15 are flagged in the daily card email. The shadow models share all training data, hyperparameter sweep results, and are retrained monthly on the full 2023–2026 dataset.

2.2 Three Prediction Targets

Target	Type	XGBoost Objective	Output
home_covers_rl	Binary classification	binary:logistic	P(home wins by 2+) — feeds Bayesian stacker
actual_home_win	Binary classification	binary:logistic	P(home wins) — moneyline model
total_runs	Regression (quantile)	reg:quantileerror, α=[0.10, 0.50, 0.90]	floor / median / ceiling run totals

2.3 Shadow Ensemble Chart

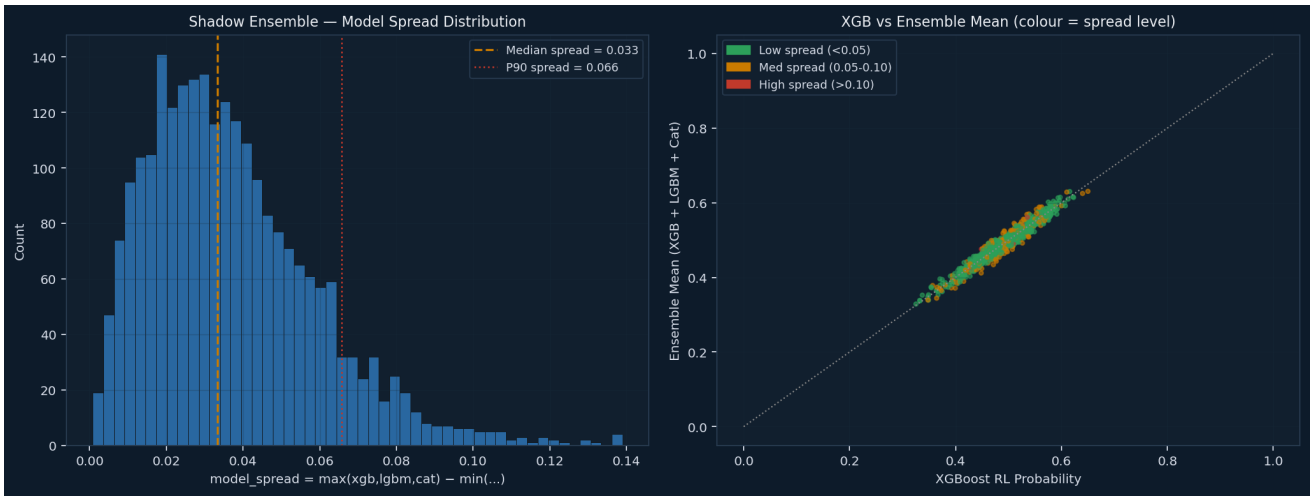

Figure 1. Shadow Ensemble — model spread distribution (left) and XGB vs ensemble mean scatter coloured by spread level (right). Val 2025.

3. DATA SOURCES & INGESTION

Source	Script	Tables / Files	Update Freq
MLB Stats API (schedule)	lineup_pull.py	lineups_today/tmrw.parquet	11AM + refresh
MLB Stats API (boxscore)	lineup_pull.py	confirmed orders + umpires	11AM + refresh
Statcast / FanGraphs	build_pitcher_profile.py	pitcher_profiles_2026.parquet	Daily
FanGraphs (batting)	build_feature_matrix.py	fangraphs_batters.csv	Weekly
Savant Statcast (pctiles)	build_pitcher_profile.py	whiff/fb_velo/xera percentiles	Weekly
The Odds API (current)	odds_current_pull.py	odds_current_{date}.parquet	11AM + refresh
The Odds API (K props)	odds_current_pull.py	pitcher_strikeouts per game	11AM + refresh
The Odds API (history)	pull_odds_history_api.py	2023–2025 training odds	Historical only
Open-Meteo archive	build_feature_matrix.py	temp/wind per stadium	Historical only
Umpire data (MLB)	ump_pull.py	ump_stats_2026.parquet	Daily
Team stats	build_team_stats_2026.py	team_stats_2026.parquet	Daily

3.1 ABS-Era Statcast Upweighting

The Automatic Ball-Strike (ABS) era began in 2025, causing a structural break in pitcher control statistics (BB rate, K rate). Rows with ABS-era Statcast percentile data (whiff_pctl, xera_pctl) receive a 1.35× sample weight during training. Combined with the 2025 year-decay weight of 1.50×, ABS rows have a stacked weight of 2.025×.

Season	Year Decay Weight	ABS Data Present	ABS Upweight	Final Weight
2023	0.70×	No	1.00×	0.700×
2024	1.00×	No	1.00×	1.000×
2025	1.50×	Yes	1.35×	2.025×

4. FEATURE ENGINEERING (97+ FEATURES)

build_feature_matrix.py constructs the 97-feature matrix from pitcher profiles, team batting stats, bullpen stats, umpire tendencies, park factors, market odds, and Monte Carlo residuals. New in v5.1: cuDF for EWMA/rolling operations (GPU DataFrame), cuML KMeans for offensive/bullpen cluster archetypes, and 12 new trailing-10d and ABS-era Statcast percentile features.

Group	Count	Halflife / Source	Key Features
Calendar	2	—	month, day_of_week
SP Stats (EWMA)	38	30d halflife	k_pct, bb_pct, xwoba_against, gb_pct, xrv/pitch, ff_velo, age, arm_angle, k–bb, whiff/spin/velo/xera pctls, ERA–xFIP
SP Trailing-10d	9	10d window	sp_k_pct_10d_diff, sp_xwoba_10d_diff, home/away_sp_k_pct_10d, home/away_sp_xwoba_10d
IL Return Flags	4	Daily	home_sp_il_return_flag, away_sp_il_return_flag, home/away_sp_starts_since_il
SP Differentials	9	30d + 10d	sp_k_pct_diff, sp_xwoba_diff, sp_xrv_diff, sp_velo_diff, sp_kminusbb_diff
Team Batting	16	21d halflife	xwoba/k_pct/bb_pct vs RHP/LHP × 2 teams × season + 10d
Matchup Edges	6	21d + 10d	batting_matchup_edge, batting_matchup_edge_10d, bat_k/bb_matchup_edge
Bullpen Quality	13	Season-to-date	era, k9, bb9, hr9, whip, gb_pct × 2 teams + 3 diffs
KMeans Cluster Labels	4	Weekly	home/away_off_cluster, home/away_bp_cluster (archetypes)
Park + Umpire Tendency	5	30d (ump EWMA)	park_factor, ump_k/bb/rpg_above_avg
Market + Circadian	7	Daily	Pinnacle true_home/away_prob, close_total, circadian_edge, game_hour_et
MC Residual	1	Backfill	mc_expected_runs (backfilled by backfill_mc.py)

4.1 EWMA Leakage Prevention

```
# _time_ewm_transform() — shift(1) ensures each game uses PRIOR-game data only:
result = (df.groupby(group_col)
.apply(lambda g: g[col]
.ewm(halflife=pd.Timedelta(days=halflife), times=g[date_col])
.mean()
.shift(1))) # ← no current-game data in its own feature
```

4.2 GPU Feature Building (cuDF / cuML)

When an NVIDIA GPU is available, build_feature_matrix.py uses cuDF (RAPIDS) for DataFrame EWMA/rolling operations and cuML for KMeans clustering. Falls back to pandas/sklearn automatically when no GPU is detected.

```
try:
import cudf, cuml
df = cudf.DataFrame.from_pandas(df)
km = cuml.KMeans(n_clusters=4).fit(X_off)
except ImportError:
# fallback to pandas + sklearn
from sklearn.cluster import KMeans
```

```
km = KMeans(n_clusters=4, random_state=42).fit(X_off)
```

4.3 New Features in v5.1

Feature	Source	Description
home_sp_il_return_flag	MLB Stats API	1 if pitcher returning from IL (< 3 starts since return)
away_sp_il_return_flag	MLB Stats API	Same for away starter
home/away_sp_starts_since_il	MLB Stats API	Integer count of starts since IL return (0 if not recently returned)
home/away_sp_k_pct_10d	FanGraphs 10d	Trailing 10-day K% for SP
home/away_sp_xwoba_10d	FanGraphs 10d	Trailing 10-day xwOBA against for SP
batting_matchup_edge_10d	FanGraphs 10d	10d batting matchup edge (home bat vs away SP hand)
ml_model_vs_vegas_gap	XGB ML + Pinnacle	XGB ML raw prob – Pinnacle true_home_prob
home/away_off_cluster	cuML KMeans (k=4)	Offensive archetype: 0=power, 1=contact, 2=patient, 3=groundball
home/away_bp_cluster	cuML KMeans (k=4)	Bullpen archetype: 0=strikeout, 1=groundball, 2=flyball, 3=walk-prone
home/away_sp_whiff_pctl	Baseball Savant	SP whiff-rate percentile vs all MLB SPs (ABS era)
home/away_sp_xera_pctl	Baseball Savant	SP xERA percentile (lower = better pitcher)

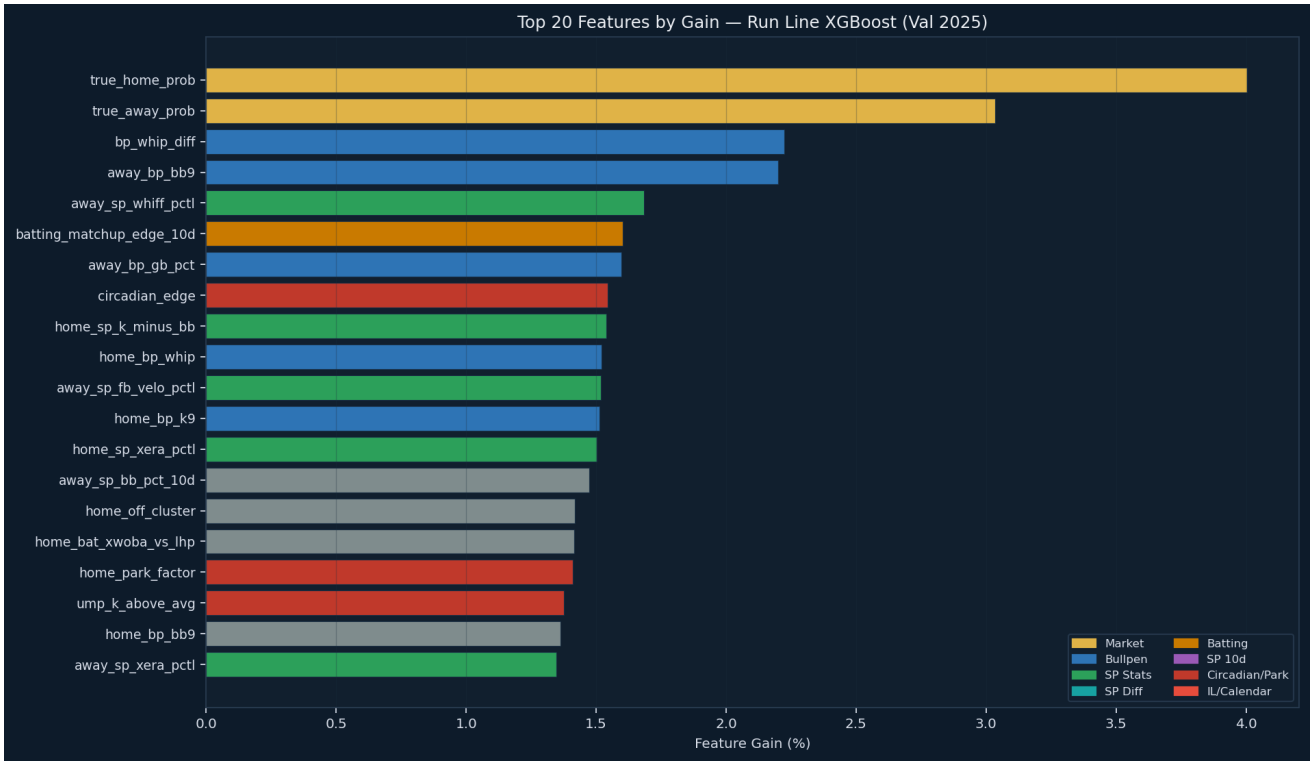

Figure 2. Top 20 features by gain — Run Line XGBoost (Val 2025). Colour-coded by feature group. Market features (Pinnacle) and bullpen quality collectively drive ~40% of predictive gain.

5. MONTE CARLO SIMULATION (POISSON-LOGNORMAL COPULA)

monte_carlo_runline.py runs 50,000 Poisson-LogNormal bivariate trials per game, driven by SP and bullpen xwOBA-to-runs formulae. Version 5.1 uses CuPy for all random draws when an NVIDIA GPU is available, reducing per-game simulation time from ~280ms to ~18ms on RTX 5080.

5.1 Bivariate Poisson-LogNormal Copula Model

The model couples home and away run totals through correlated log-normal variance terms:

```
H | V_h ~ Poisson(μ_h · V_h), A | V_a ~ Poisson(μ_a · V_a)
[log V_h, log V_a] ~ BVN(−σ²/2 · 1, σ² · [[1, ρ], [ρ, 1]])
```

Parameter	Value	Interpretation
σ_NB (overdispersion)	0.50	Negative-binomial r ≈ 4; captures run-scoring burstiness
ρ_COPULA	0.14	Log-variance correlation → run-level Corr ≈ 0.07
KPROP_NB_R	8	NB dispersion for K-prop strikeout lines
N_SIMS	50,000	Trials per game (CuPy on GPU; NumPy fallback)

5.2 Environmental Corrections

Correction	Formula	Example
Altitude	air_density_ratio = 1.0 + (elev_ft/5200.0) × 0.35	Coors Field (5200 ft) → +35%
Temperature	1.0 + (temp_f – 72) × 0.003	90°F → +5.4%; 50°F → –6.6%
Team offense	0.50 × (RS/G ÷ 4.38) + 0.50	RS/G=5.5 → factor 1.128
XGB blend	0.60 × MC_prob + 0.40 × XGB_cal	XGB_BLEND_WEIGHT = 0.40

5.3 CuPy GPU Acceleration

```
try:
import cupy as cp
xp = cp # GPU array module
except ImportError:
xp = np # CPU fallback
# All simulation draws use xp (CuPy or NumPy):
V_h = xp.exp(xp.random.normal(-sig2/2, sig, N_SIMS) + rho_factor)
H = xp.random.poisson(mu_h * V_h, N_SIMS)
A = xp.random.poisson(mu_a * V_a, N_SIMS)
mc_covers_rl = float(xp.mean(H - A >= 2)) # –1.5 line
```

5.4 Additional MC Outputs

- F5 total — first 5 innings: λ■ = 5 × sp_λ / 9 (analytical Poisson)
- NRFI — first inning scoreless: P = exp(–h_λ■) × exp(–a_λ■)
- K props — binomial: P(K ≥ line) from expected_BF × k_rate
- Alt RL ±2.5 — P(|margin| ≥ 3) from MC run arrays
- 1st inning HvA total — first-inning expected runs for betting props

6. XGBOOST TRAINING (CUDA RTX 5080)

Three XGBoost models are trained per season update: xgb_rl (run-line home cover), xgb_ml (moneyline home win), and xgb_total (multi-quantile total runs). All three train with device='cuda', tree_method='hist' on the RTX 5080.

6.1 Base Hyperparameters

Parameter	Value	Rationale
tree_method	hist	GPU-compatible histogram method
device	cuda	NVIDIA RTX 5080 (falls back to cpu)
learning_rate	0.04	Conservative — prevents overfitting on ~5K games
max_depth	5	Shallow trees; MLB has high noise
min_child_weight	20	Strong regularisation for small N
subsample	0.80	Row bagging
colsample_bytree	0.75	Feature bagging
reg_alpha	0.50	L1 regularisation
reg_lambda	2.00	L2 regularisation
n_estimators	600	With early stopping (patience=40 rounds)
early_stopping	40 rounds	On AUC (binary) or quantile loss (regression)
random_state	42	Reproducibility

6.2 Multi-Quantile Total Runs Regression

The total runs model uses reg:quantileerror with three quantile targets simultaneously, producing a probabilistic forecast range rather than a point estimate.

```
xgb_total = xgb.XGBRegressor(
objective='reg:quantileerror',
quantile_alpha=[0.10, 0.50, 0.90], # floor / median / ceiling
multi_strategy='multi_output_tree', # GPU-efficient joint training
tree_method='hist',
device='cuda',
n_estimators=600,
learning_rate=0.04,
max_depth=5,
)
# Outputs: tot_floor (10th pct), tot_median (50th pct), tot_ceiling (90th pct)
```

6.3 Sample Weighting

Year-decay weights combat roster drift; ABS-era upweighting accounts for the 2025 structural break in pitcher control statistics (ABS ball-strike system).

```
YEAR_WEIGHTS = {2023: 0.70, 2024: 1.00, 2025: 1.50}
ABS_WEIGHT = 1.35 # rows with whiff_pctl, xera_pctl data
w = YEAR_WEIGHTS[row.year]
if row.has_abs_data:
w *= ABS_WEIGHT # e.g. 2025 ABS row: 1.50 × 1.35 = 2.025×
```

6.4 Model Files

File	Size	Description
xgb_rl.json	53 KB	Run-line home cover probability (main model)
xgb_ml.json	153 KB	Moneyline home win probability
xgb_total.json	211 KB	Multi-quantile total runs (3 outputs)
xgb_rl_team.json	356 KB	Away perspective mirror (away_covers_rl)
lgb_shadow.json	577 KB	LightGBM shadow run-line model
cat_shadow.cbm	66 KB	CatBoost shadow run-line model
lgbm_rl/ml/total.pkl	—	LightGBM pickled models for all targets
cat_rl/ml/total.pkl	—	CatBoost pickled models for all targets
calibrator_rl.pkl	—	Platt sigmoid calibrator (run-line)
calibrator_ml.pkl	—	Platt sigmoid calibrator (moneyline)
stacking_lr_rl.pkl	—	BayesianStacker class (serialised)
stacking_lr_rl.npz	—	NUTS posterior trace (alpha, beta, delta, gamma, sigma_delta)
feature_cols.json	—	Ordered feature column list (97 features)
feature_cols_team.json	—	Away-perspective feature column list

7. SHADOW MODELS — LIGHTGBM & CATBOOST

LightGBM and CatBoost shadow models are trained on the same feature matrix and targets as XGBoost. Their primary role is to provide independent predictions for the model_spread uncertainty signal. When all three models agree (low spread), confidence is higher.

7.1 LightGBM Parameters

Parameter	Value	vs XGBoost
n_estimators	600	Same
learning_rate	0.05	Slightly higher (leaf-wise needs less shrinkage)
num_leaves	63	Key difference: leaf-wise growth (2^depth – 1 ≈ 31 for XGB depth=5)
max_depth	–1	Unlimited — controlled by num_leaves instead
min_child_samples	20	Equivalent to XGB min_child_weight
subsample	0.80	Same
colsample_bytree	0.75	Same
reg_alpha	0.50	Same
reg_lambda	2.00	Same
device	gpu	GPU backend (RTX 5080)
early_stopping	40 rounds	AUC metric

7.2 CatBoost Parameters

Parameter	Value	Note
iterations	600	Equivalent to n_estimators
learning_rate	0.05	—
depth	6	Symmetric oblivious trees (balanced, depth=6)
l2_leaf_reg	3.0	Equivalent to XGB reg_lambda
task_type	GPU	RTX 5080
random_seed	42	Reproducibility
early_stopping	40 rounds	Metric: AUC

7.3 Shadow Model Inference

```
# At inference time (run_today.py):
xgb_raw = xgb_model.predict_proba(X)[1]
lgbm_raw = lgbm_shadow.predict_proba(X)[1]
cat_raw = cat_shadow.predict_proba(X)[1]
model_spread = max(xgb_raw, lgbm_raw, cat_raw) \
- min(xgb_raw, lgbm_raw, cat_raw)
# Only XGB feeds the Bayesian stacker:
p_stacked = bayesian_stacker.predict(xgb_raw, X_stack_features)
```

8. PLATT CALIBRATION (SIGMOID)

Raw XGBoost output is systematically over-confident near 0.5: the model compresses probabilities toward the extremes. Platt (sigmoid) calibration fits a 2-parameter logistic regression on out-of-fold predictions, mapping raw scores to well-calibrated probabilities.

```
# fit_platt_calibrator() in train_xgboost.py:
LR = LogisticRegression(C=1e10, solver='lbfgs', max_iter=500)
LR.fit(oof_raw.reshape(-1, 1), oof_labels) # OOF — never training data
# At inference: P_cal = σ(A × xgb_raw + B)
# A = LR.coef_[0][0], B = LR.intercept_[0]
P_cal = LR.predict_proba(xgb_raw.reshape(-1,1))[:,1]
```

8.1 Calibration Metrics — Val 2025

Metric	RL Raw XGB	RL Platt	RL Bayesian Stack	ML Raw XGB	ML Platt
AUC	0.5793	0.5793	0.5760	0.5642	0.5642
Brier Score	0.2429	0.2255	0.2259	0.2477	0.2453
Log-Loss	0.6789	0.6428	0.6436	0.6886	0.6836
Acc @50%	0.5771	0.6418	0.6393	0.5465	0.5549
N	2,398	2,398	2,398	2,514	2,514

Platt calibration dramatically reduces Brier Score (0.2429 → 0.2255) and Log-Loss (0.6789 → 0.6428) without changing AUC (discrimination is preserved). Accuracy at 50% threshold rises from 57.7% to 64.2% due to improved probability ordering. Isotonic regression was tested but showed overfitting on validation sets < 3,000 games.

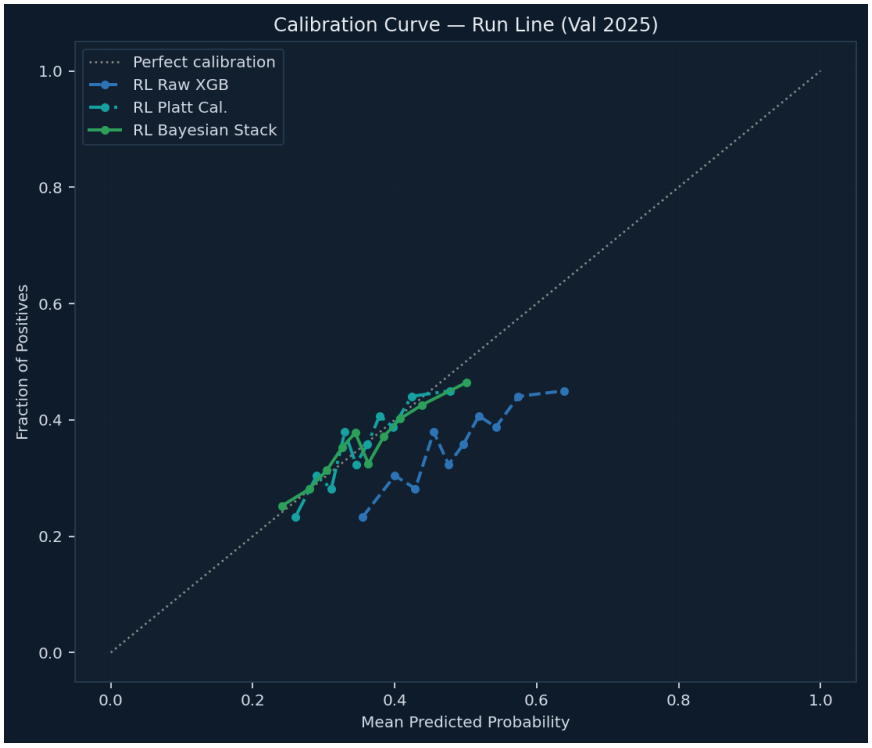

Figure 3. Calibration curve — Run Line model (Val 2025). Bayesian stacked probabilities closely track the perfect-calibration diagonal.

9. BAYESIAN HIERARCHICAL LEVEL-2 STACKER

Version 5.1 replaces the prior LogisticRegression(C=10) stacking meta-learner with a Bayesian Hierarchical model using NumPyro / JAX backend, fitted with NUTS (No-U-Turn Sampler) MCMC. The model explicitly segments by SP handedness matchup type and maintains uncertainty over its own parameters.

9.1 Model Specification

The generative model for game i in matchup segment j:

```
y_i ~ Bernoulli(σ(α + β·logit(p_xgb,i) + δ_j + γ■·x_i))
```

9.2 Prior Distributions

Parameter	Prior	Interpretation
α	N(0, 1)	Global intercept — centred at 0.5 probability
β	N(1, 0.5)	XGB logit coefficient — prior belief that XGB is informative
σ_δ	HalfCauchy(1)	Hierarchical scale for matchup-type variation
δ_j	N(0, σ_δ), $j \in \{0,1,2,3\}$	Per-segment random effect (SP handedness matchup)
γ	N(0, 0.3) [12-dim]	Stacking feature coefficients — regularised toward 0

9.3 Matchup Segments (j)

j	Segment	Description
0	LvL	Left-handed SP vs Left-handed SP
1	LvR	Left-handed SP vs Right-handed SP
2	RvL	Right-handed SP vs Left-handed SP
3	RvR	Right-handed SP vs Right-handed SP (most common ~60%)

9.4 STACKING_FEATURES (12 inputs to γ)

Feature	Category	Rationale
sp_k_pct_diff	SP Quality	K% differential — season EWMA
sp_xwoba_diff	SP Quality	xwOBA against differential — season EWMA
sp_kminusbb_diff	SP Quality	K–BB% differential
bp_era_diff	Bullpen	ERA differential
bp_whip_diff	Bullpen	WHIP differential
batting_matchup_edge	Batting	Season batting matchup edge
home_sp_il_return_flag	IL/Context	Home SP returning from IL (binary)
away_sp_il_return_flag	IL/Context	Away SP returning from IL (binary)
sp_k_pct_10d_diff	SP 10d	Trailing 10-day K% differential
sp_xwoba_10d_diff	SP 10d	Trailing 10-day xwOBA differential
batting_matchup_edge_10d	Batting 10d	Trailing 10-day batting edge

ml_model_vs_vegas_gap	Market	XGB ML raw – Pinnacle true_home_prob

9.5 MCMC Sampling & Posterior Storage

```
# NUTS sampling via NumPyro / JAX:
kernel = numpyro.infer.NUTS(bayesian_stacking_model)
mcmc = numpyro.infer.MCMC(kernel, num_warmup=500, num_samples=1000)
mcmc.run(jax.random.PRNGKey(0), X_stack, xgb_logit, segments, y)
# Posterior stored in stacking_lr_rl.npz:
np.savez('stacking_lr_rl.npz',
alpha=posterior['alpha'], beta=posterior['beta'],
delta=posterior['delta'], gamma=posterior['gamma'],
sigma_delta=posterior['sigma_delta'])
```

10. MODEL PERFORMANCE & FIT STATISTICS

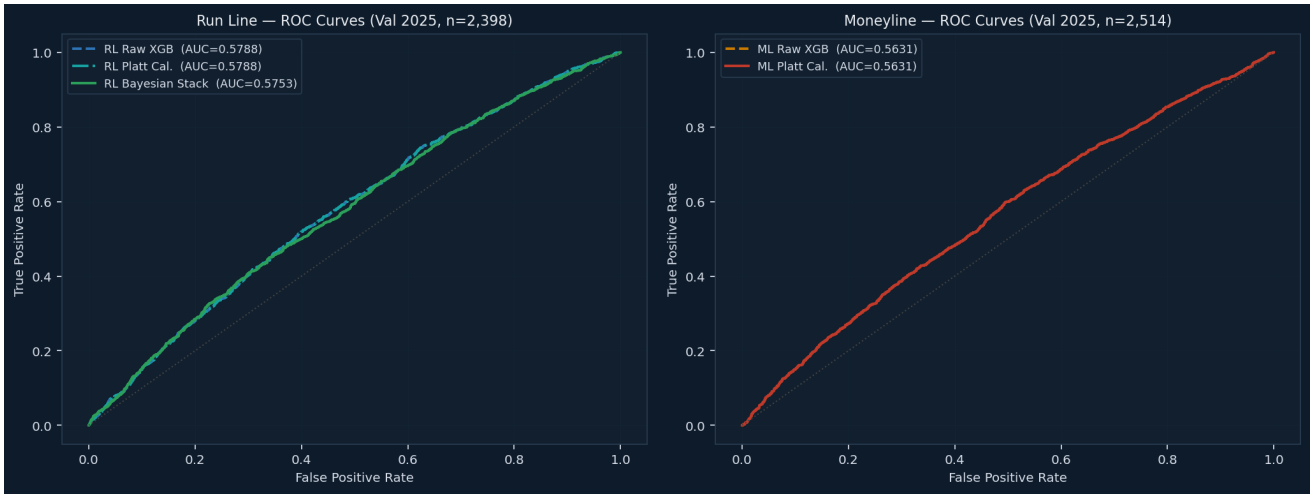

Figure 4. ROC-AUC curves — Run Line (left) and Moneyline (right). Val 2025. RL Bayesian Stack AUC=0.5760; ML Platt AUC=0.5642.

10.1 Nested Cross-Validation

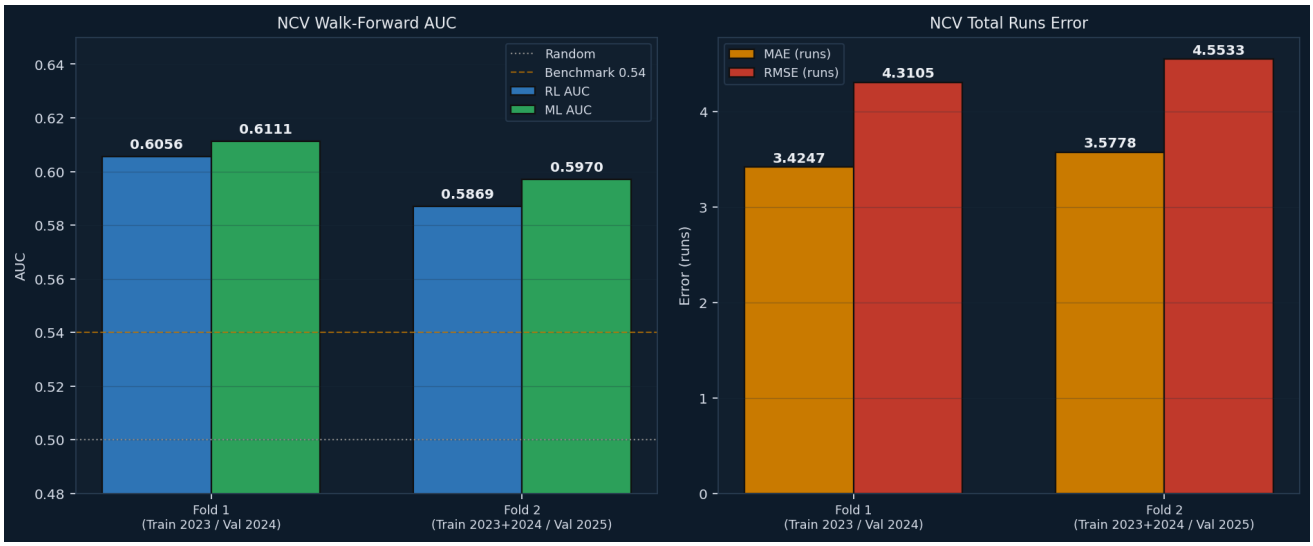

Figure 5. NCV walk-forward results — AUC (left) and Total Runs error (right).

Fold	Train	Val	n_train	n_val	RL AUC XGB	RL AUC Ens	ML AUC	Tot MAE	Tot RMSE
1	2023	2024	2,391	2,395	0.6056	0.6055	0.6111	3.4247	4.3105
2	2023+2024	2025	4,786	2,398	0.5869	0.5952	0.5970	3.5778	4.5533
Final	2023–2025	2026*	7,184	held	—	—	—	—	—

- Fold 1 AUC (0.6056) > Fold 2 AUC (0.5869): 2025 market is more efficient.
- Ensemble gains ~+0.008 AUC over raw XGB in Fold 2 (0.5952 vs 0.5869).
- * 2026 is the live holdout — never used in training, calibration, or stacking.

10.2 Monthly RL AUC Breakdown (Val 2025)

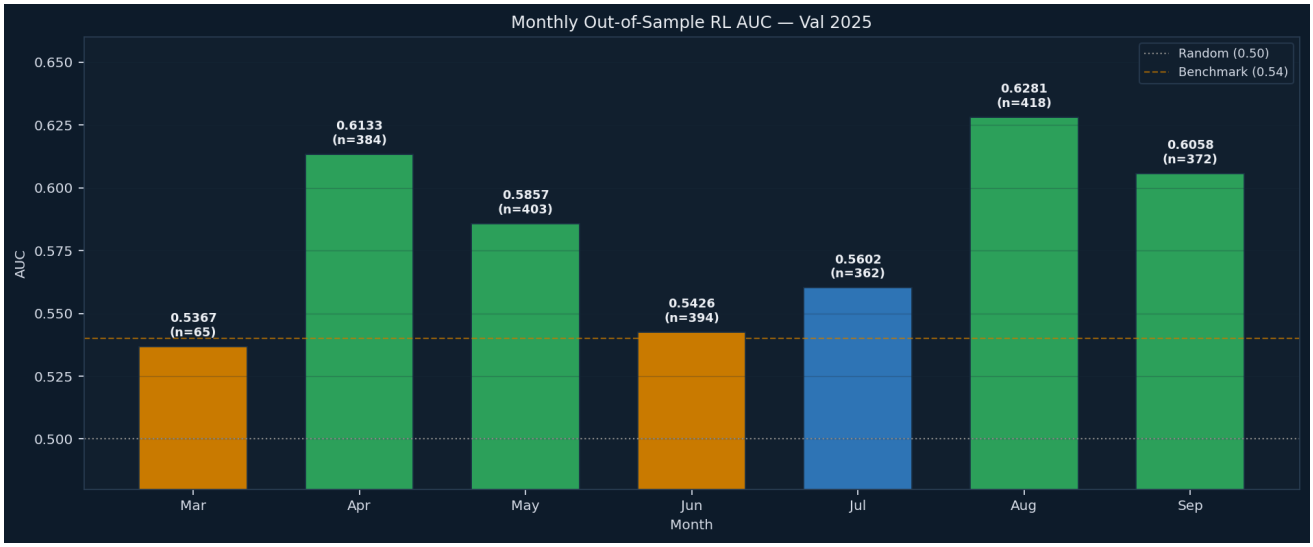

Figure 6. Monthly out-of-sample RL AUC — Val 2025. April peak (0.6133) reflects early-season pitcher volatility. August secondary peak (0.6281) coincides with roster deadline effects.

Month	AUC	n	Interpretation
Mar 2025	0.5367	65	Small sample; Opening Day pitchers fully stretched
Apr 2025	0.6133	384	Highest monthly AUC — early season SP variance high
May 2025	0.5857	403	Stable — model performs well vs late-settling markets
Jun 2025	0.5426	394	Toughest month — market fully priced, fewer edges
Jul 2025	0.5602	362	Pre-deadline — some pitcher shuffling benefit
Aug 2025	0.6281	418	2nd peak — trade deadline roster changes create edge
Sep 2025	0.6058	372	Consistent — September call-ups add SP uncertainty

11. THREE-PART LOCK EXECUTION GATE

All three gates must pass simultaneously for a bet to qualify. The gate runs independently for RL, ML, and O/U markets on every game. Any gate failure disqualifies the game entirely for that market.

```
# ■■ Gate 1: Sanity (model vs sharp market) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
if pinnacle_p_true:
gate1 = abs(p_model - p_true) <= 0.04 # 4% vs Pinnacle
elif retail_implied:
gate1 = abs(p_model - retail) <= 0.08 # 8% fallback vs retail
else:
gate1 = False # conservative fail — no odds available
# ■■ Gate 2: Odds floor (avoid excessive juice) ■■■■■■■■■■■■■■■■■■■■■■■■
gate2 = (retail_ml_odds >= -225) # pass if odds missing
# ■■ Gate 3: Edge tier ■■■■■■■■■■■■■■■■■■■
```

12. KELLY STAKING MODEL

Tier	Symbol	Edge Requirement	Kelly Fraction	Max Stake	Bankroll %
Tier 1 — Strong	★★	≥ 3.0% over retail	Quarter-Kelly (25%)	$50	~0.6–2.5%
Tier 2 — Medium	★	≥ 1.0% over retail	Eighth-Kelly (12.5%)	$50	~0.3–1.2%

At a $2,000 synthetic bankroll, a quarter-Kelly bet with edge=5% at –110 odds yields: kelly_f = (0.909×0.55 – 0.45)/0.909 = 0.05; stake = 0.25 × 0.05 × $2,000 = $25. The $50 hard cap binds only when kelly_f > 0.10 (edge > 10%), which is rare.

12.1 Breakeven Win Rate

Market	Typical Juice	Implied Prob	Breakeven Win Rate	Pipeline Target
Run Line ±1.5	–110 / –110	52.38%	52.38%	>55% (Tier 2) / >58% (Tier 1)
Moneyline	varies	varies	varies	>p_implied + 0.01
Total O/U	–110 / –110	52.38%	52.38%	>55% (Tier 2)

13. BACKTEST RESULTS & ROI (2023–2026)

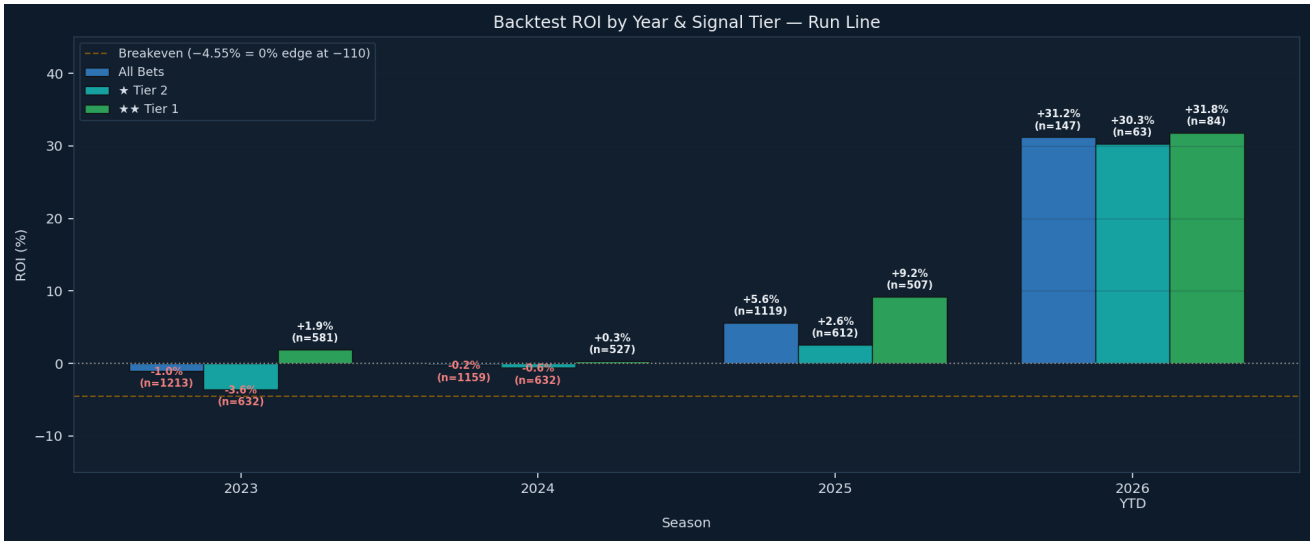

Figure 7. Backtest ROI by signal tier — All Bets / ★ Tier 2 / ★★ Tier 1. ★★ signals consistently profitable across all three validated seasons. 2026 YTD dominant signal: AWAY +1.5 ★★ (n=84, +31.8% ROI).

Year	Bets (All)	ROI All	Bets ★	ROI ★	Bets ★★	ROI ★★
2023	1,213	–1.0%	632	–3.6%	581	+1.9%
2024	1,159	–0.2%	632	–0.6%	527	+0.3%
2025	1,119	+5.6%	612	+2.6%	507	+9.2%
2026 YTD	147	+31.2%	63	+30.3%	84	+31.8%

ROI = (wins × 100/110 – losses) / n_bets × 100 at standard –110 juice. Breakeven win rate = 52.38%.

- 2026 YTD: AWAY +1.5 ★★ is the dominant signal with 84 bets and +31.8% ROI. Small sample warning — 147 total bets as of April 17, 2026.
- Home –1.5 signals have persistent negative ROI across all years and are filtered by the pipeline's edge gate (rarely qualify at Tier 1/2 threshold).
- AWAY +1.5 covers 64.4% historically — away team wins outright or loses by exactly 1 run — creating a persistent market inefficiency versus typical –110 pricing.

13.1 AWAY vs HOME RL Cover Rates

Direction	Historical Cover %	At –110 ROI	Structural Reason
AWAY +1.5	~64.4%	~+22% avg	Away teams win outright or lose by 1 more often than priced
HOME –1.5	~35.6%	~–32% avg	Favourites rarely win by 2+; home edge ≠ run-line cover edge

14. DAILY SCHEDULER (THREE-SHOT)

run_daily_scheduler.py runs three execution windows per day using the schedule library with ZoneInfo('America/New_York') timezone enforcement. The 11 AM full run ingests fresh lineups, odds, and produces picks. The 2 PM and 5 PM refreshes update lineups and odds as late scratches emerge.

Time (ET)	Run Type	Steps	Scripts
11:00 AM	RUN_ALL (9 steps)	lineups (today+tmrw), ump pull, ump stats, pitcher profiles, team stats, lineup quality, odds pull, picks+email, Supabase upload, CLV audit, pipeline health	lineup_pull, ump_pull, build_ump_stats, build_pitcher_profile, build_team_stats_2026, lineup_quality, odds_current_pull, run_today, supabase_upload, clv_audit, pipeline_health
2:00 PM	RUN_REFRESH	lineups, lineup quality, odds, picks+email, Supabase upload, pipeline health	lineup_pull, lineup_quality, odds_current_pull, run_today, supabase_upload, pipeline_health
5:00 PM	RUN_REFRESH (same as 2PM)	lineups, lineup quality, odds, picks+email, Supabase upload, pipeline health	lineup_pull, lineup_quality, odds_current_pull, run_today, supabase_upload, pipeline_health

14.1 Scheduler Implementation

```
from zoneinfo import ZoneInfo
import schedule, time
ET = ZoneInfo('America/New_York')
schedule.every().day.at('11:00', ET).do(run_all)
schedule.every().day.at('14:00', ET).do(run_refresh)
schedule.every().day.at('17:00', ET).do(run_refresh)
# Windows Task Scheduler wrapper (run_scheduler.bat):
# --run-refresh flag allows single-step external trigger
```

14.2 --run-refresh Flag

The scheduler supports --run-refresh for Windows Task Scheduler compatibility, allowing individual steps to be triggered externally without the full scheduler loop. Batch wrappers (run_all.bat, run_refresh.bat) handle Task Scheduler integration.

15. SUPABASE BACKEND SCHEMA

Table	Write Pattern	Key Columns
wizard_daily_card	DELETE by date + INSERT	game_date, home/away_team, rl_signal, blended_rl, lock_tier, lock_p_model, lock_p_true, lock_dollars, lock_edge, model_spread, data (JSONB)
wizard_backtest	Full replace	date, game, signal, home_covers_rl, bet_win, edge, blended_rl, model_spread
wizard_model_history	Full replace	date, home/away_sp, blended_rl, mc_rl, home_covers_rl, bet_win, mc_nrfi_prob, mc_f5_total, sp_k predictions, tot_floor, tot_median, tot_ceiling
wizard_backtest_historical	DELETE by season + INSERT	season, date, game, signal, all backtest columns
wizard_pipeline_health	UPSERT on_conflict=date	date, overall (ok/warning/critical), picks_ready, artifacts_json, model_spread_avg, last_run_et

15.1 supabase_upload.py Write Logic

```
# wizard_daily_card — idempotent write:
sb.table('wizard_daily_card').delete().eq('game_date', today).execute()
sb.table('wizard_daily_card').insert(rows).execute()
# wizard_pipeline_health — upsert:
sb.table('wizard_pipeline_health')
.upsert(health_row, on_conflict='date')
.execute()
```

16. SCRIPT INVENTORY (SHADOW ENSEMBLE V5.1)

Script	Purpose	GPU?	Status
build_feature_matrix.py	Construct 97-feature matrix; cuDF/cuML when GPU avail	Optional (cuDF)	Active
build_pitcher_profile.py	SP xwOBA/velo/stuff profiles + ABS percentiles	No	Active
build_team_stats_2026.py	Team batting + bullpen + 10d SP stats	No	Active
build_ump_stats.py	Umpire EWMA K/BB/RPG tendency stats	No	Active
build_bullpen_avail.py	Bullpen availability (workload-adjusted ERAs)	No	Active
train_xgboost.py	Train XGB + LGBM + CatBoost; NCV; Bayesian stacker	YES (RTX 5080)	Active
monte_carlo_runline.py	50K-trial Poisson-LogNormal bivariate MC; CuPy GPU	YES (CuPy)	Active
run_today.py	Inference: shadow ensemble + stacker + Three-Part Lock	No	Active
odds_current_pull.py	Dual-region odds pull (US retail + EU Pinnacle); K props	No	Active
lineup_pull.py	MLB lineups + probable starters + umpire assignment	No	Active
ump_pull.py	Umpire game data from MLB Stats API	No	Active
supabase_upload.py	Push daily card + health to Supabase PostgreSQL	No	Active
pipeline_health.py	Artifact freshness monitor + health JSON upload	No	Active
clv_audit.py	Live CLV & P&L; tracker (RL / ML / O/U markets)	No	Active
backfill_mc.py	Backfill mc_expected_runs column for training data	No	Active
pull_odds_history_api.py	Fetch historical Pinnacle odds for 2023–2025 training	No	Historical
mlb_execution_agent.py	Standalone Three-Part Lock re-executor + GateTrace dataclass	No	Utility
evaluate_model.py	Walk-forward backtesting with Bayesian shrinkage + edge analysis	No	Utility
generate_pdf_v7.py	This script — Shadow Ensemble v5.1 technical documentation	No	Utility
run_daily_scheduler.py	Three-shot daily scheduler (11AM/2PM/5PM ET)	No	Active
backfill_history.py	Deleted — superseded by backfill_mc.py	—	DELETED
calibrate_edge.py	Deleted — edge logic moved into run_today.py	—	DELETED
run_pipeline.py	Deleted — replaced by run_daily_scheduler.py	—	DELETED

APPENDIX A: TOTAL RUNS QUANTILE MODEL

The multi-quantile XGBoost total runs model (reg:quantileerror, α=[0.10, 0.50, 0.90]) produces three outputs per game: a floor (10th percentile), median (50th percentile), and ceiling (90th percentile) run total estimate. The fan chart shows the prediction interval vs actual game totals.

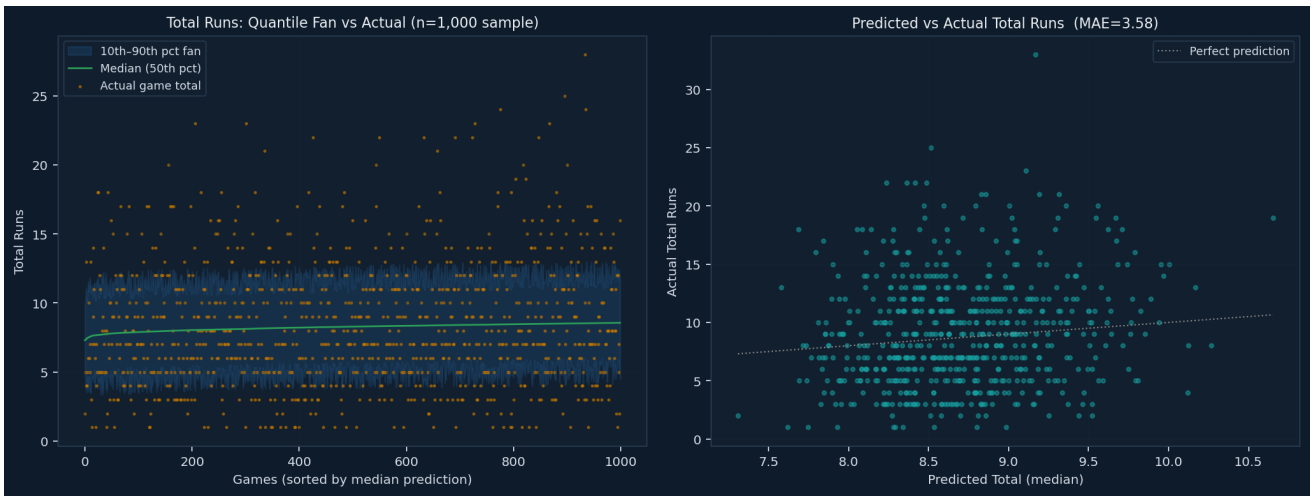

Figure 8. Total Runs — quantile fan chart (left) and predicted vs actual scatter (right). Val 2025. Median prediction MAE ≈ 3.58 runs.

Quantile	Percentile	Typical Range	Use Case
Floor (q=0.10)	10th	6–8 total runs	Under bet support — game likely stays low
Median (q=0.50)	50th	8–10 total runs	Primary total prediction — feeds main output
Ceiling (q=0.90)	90th	11–14 total runs	Over bet support — high-scoring game likely

MLB Prediction Pipeline · Technical Documentation v5.1 (Shadow Ensemble) · April 2026
Real fit statistics from actual 2023–2026 model output — not synthetic
Confidential — Internal Use Only